Mathematical Foundations

Emission-Trajectory ML Project

Dr. Khawar Naeem
Qatar Transportation and Traffic Safety Center (QTTSC), Qatar University

14 July 2026 (living document; grows with each modeling session)

Abstract

This document formalizes the mathematics behind the emission-trajectory forecasting pipeline (`https://github.com/khawar89/energy-carbon-forecast-ml`): the evaluation metrics every model is scored by, the two baselines that define the bar a model must clear, Ridge regression as the project's first learned model, the ensemble and boosting methods that follow it, and the uncertainty quantification that decides whether a skill difference is statistically real. Each equation is numbered and tied directly to the corresponding code in `src/evaluate.py`, `notebooks/03_baselines.ipynb`, `notebooks/04_models_ridge_tree.ipynb`, and `notebooks/05_xgboost.ipynb`. New sections are added as later sessions (error analysis) are built, so this remains a growing companion to the coding sessions rather than a one-time summary. This is intended to support future methodology writing and research extension of the project, not to substitute for it.

Contents

1 Notation

This document uses consistent notation across all sections, matching the pipeline built in `src/build_features.py` and `src/evaluate.py`.

- c indexes a country; t indexes a calendar year.
- $y_{c,t}$ denotes the observed production-based CO_2 emissions (Mt) of country c in year t.
- The *feature year* t is the year whose information is known at prediction time; the *target year* $t+1$ is the year being predicted. No feature used to predict $y_{c,t+1}$ may use information from after year t (see the project's `CLAUDE.md` and `AGENTS.md` for the full leakage-prevention rationale; every equation in this document assumes that constraint is already enforced).
- $\hat{y}_{c,t+1}$ denotes a model's prediction of $y_{c,t+1}$, made using information available through year t.
- $x_{c,t} \in \mathbb{R}^p$ denotes the feature vector for country c at year t (lags, rolling statistics, population, and related columns), with p the number of features.
- n denotes the number of (country, year) rows in a given evaluation set (for example, the validation split).
- Splits are labeled train, val (validation), and test, assigned by the *target* year $t+1$, not the feature year t: train covers target years through 2014, val covers 2015–2018, test covers 2019–2023, and a separate provisional table covers target year 2024.

2 Evaluation Metrics

Every model in this project, from the persistence baseline through XGBoost, is scored by the same set of metrics, implemented once in `src/evaluate.py` and reused unchanged in every later session. This section states each metric formally.

2.1 Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i| \tag{1}$$

MAE is the average size of a miss, in the original units (Mt CO_2). It is interpretable, but because errors are measured in absolute units, its value is dominated by the largest countries by emissions level.

2.2 Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2} \tag{2}$$

Squaring each error before averaging penalizes large misses quadratically. Consequently RMSE $\gg$ MAE for a given model signals that its error budget is concentrated in a small number of large misses (a "fat tail") rather than spread evenly across rows.

2.3 Median Absolute Error (MedianAE)

$$\text{MedianAE} = \text{median}_i \, |y_i - \hat{y}_i| \tag{3}$$

MedianAE is immune to the largest emitters: it reports the typical error for the typical country. Comparing Equations 1 and 3 on the same model reveals how much of the average error is attributable to a handful of very large countries rather than to the rest of the sample, a scale-dependence issue discussed generally by [10].

2.4 Median Absolute Percentage Error (MdAPE)

$$\text{MdAPE} = 100 \times \text{median}_{i\,:\,y_i \geq \tau} \left(\frac{|y_i - \hat{y}_i|}{y_i} \right) \tag{4}$$

where τ is a minimum-level threshold (this project uses $\tau = 1$ Mt). Percentage errors are undefined or explosive as $y_i \to 0$; the exclusion in Equation 4 is part of the metric's definition, not an afterthought, following the general caution around percentage-based accuracy measures raised by [10].

2.5 Persistence Skill Score

$$\text{Skill} = 1 - \frac{\text{MAE}_{\text{model}}}{\text{MAE}_{\text{persistence}}} \tag{5}$$

The skill score in Equation 5 measures a model's error relative to the persistence baseline (Section 3), rather than in absolute terms. Skill = 0 means the model is exactly as accurate as assuming no change; Skill = 1 is the unreachable limit of zero error; negative skill means the model is less accurate than doing nothing and must be reported as a loss, not merely a smaller win.

2.6 Level and delta parameterizations, scored on the level

A model in this project may be trained against either of two targets: the level $y_{c,t+1}$ directly, or the year-over-year change $\delta_{c,t+1} = y_{c,t+1} - y_{c,t}$. In the delta parameterization the model's raw output $\hat{\delta}_{c,t+1}$ is converted back to a level before any metric is computed:

$$\hat{y}_{c,t+1} = \begin{cases} \hat{f}(x_{c,t}) & \text{(level parameterization)} \\ y_{c,t} + \hat{\delta}_{c,t+1} & \text{(delta parameterization)} \end{cases} \tag{6}$$

All metrics in this section are always computed on the reconstructed level of Equation 6, on identical rows, so the two parameterizations remain directly comparable. The motivation is that next-year emissions are close to this-year emissions (Section 3), so a level-target model can appear accurate by approximately reproducing its own `co2` input; predicting the change forces the model to learn what moves emissions. The parameterization changes the question the model is asked, never the quantity it is scored on.

2.7 The same-rows rule

Every comparison table in this project scores all candidate models on an identical set of rows: a row missing any model's prediction, or missing the target itself, is dropped for every model before Equations 1–5 are computed (`evaluate.comparison_table`). Without this rule, a model could appear to win only because it was scored on an easier subset of rows.

3 Baselines

A baseline is a model that requires no fitting: no parameters are estimated from data. Its purpose is to define the bar every learned model must clear (Section 2's skill score is measured against it).

3.1 Persistence

$$\hat{y}_{c,t+1} = y_{c,t} \tag{7}$$

Equation 7 predicts that next year's level equals this year's level. This is the naive forecast, and it is the optimal forecast under a random-walk-without-drift model of the underlying series [9]. It is a strong baseline for country-level CO_2 specifically because the underlying physical stock (power plants, vehicle fleets, industrial capital) changes slowly: the median absolute year-over-year change in this dataset since 2010 is approximately 4.4% of the level (Session 1 EDA).

3.2 Linear Trend (Drift)

$$\hat{y}_{c,t+1} = y_{c,t} + \bar{\Delta}^{(5)}_{c,t}, \qquad \bar{\Delta}^{(5)}_{c,t} = \frac{y_{c,t} - y_{c,t-5}}{5} \tag{8}$$

Equation 8 extrapolates the average yearly change over the preceding five years one step forward. It is this project's five-year-windowed variant of the classical drift method [9], which in its general form draws a line between a series' first and last observed points and extrapolates it. The trade-off is explicit: Equation 8 outperforms Equation 7 for a country on a sustained trajectory, and underperforms it the moment a country's direction reverses, since the linear extrapolation has no mechanism to anticipate a turn.

3.3 Bias direction

For any series with a genuine sustained trend, persistence (Equation 7) is a biased estimator of $y_{c,t+1}$: if $y_{c,t}$ is rising, persistence systematically under-predicts; if $y_{c,t}$ is falling (for example, a country whose emissions peaked and have since declined), persistence systematically over-predicts. The sign of the bias tracks the sign of the country's own recent trend, not a fixed direction.

4 Ridge Regression

4.1 Ordinary least squares

Given a design matrix $X \in \mathbb{R}^{n \times p}$ (rows are (country, year) observations, columns are features) and target vector $y \in \mathbb{R}^n$, ordinary least squares (OLS) solves

$$\hat{\beta}_{\text{OLS}} = \arg\min_{\beta} \|y - X\beta\|_2^2 \tag{9}$$

with closed-form solution $\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y$, provided $X^\top X$ is invertible.

4.2 The problem with correlated features

This project's feature set includes multiple lags and rolling statistics of the same underlying series (`co2_lag1`, `co2_lag3`, `co2_lag5`, `co2_lag10`, `co2_roll5_mean`, `co2_roll10_mean`), which are, by construction, highly correlated with one another. When columns of X are strongly correlated, $X^\top X$ is close to singular, and $\hat{\beta}_{\text{OLS}}$ becomes highly sensitive to small changes in the data: its variance inflates even though it remains unbiased.

4.3 Ridge regression

Ridge regression adds an ℓ_2 penalty on the coefficients:

$$\hat{\beta}_{\text{ridge}} = \arg\min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2, \qquad \alpha \geq 0 \tag{10}$$

with closed-form solution

$$\hat{\beta}_{\text{ridge}} = \left(X^\top X + \alpha I\right)^{-1} X^\top y \tag{11}$$

Adding αI to $X^\top X$ in Equation 11 guarantees invertibility for any $\alpha > 0$ and shrinks the coefficients of correlated features toward one another and toward zero. This trades a small amount of bias for a reduction in variance [8], and is the reason this project prefers Ridge over OLS given its correlated lag and rolling-statistic feature set.

4.4 Preprocessing

Because the ridge penalty in Equation 10 is applied uniformly to all coefficients, features must be on comparable scales before fitting; unlike tree-based models (Section 5), Ridge requires feature scaling, and that scaler must be fit on training rows only, to avoid leaking validation or test distributional information into its parameters.

5 Ensembles and Boosting

5.1 Regression trees

A regression tree partitions the feature space into disjoint regions $R_1, \ldots, R_J$ and predicts the mean target value within each region. A single split at feature j, threshold s is chosen to minimize the total sum of squared residuals across the two resulting regions:

$$(j^*, s^*) = \arg\min_{j,s} \left[\sum_{i:\, x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2 \right] \tag{12}$$

Trees require no feature scaling: Equation 12 depends only on the ordering of feature values within a column, not their magnitude, unlike the Ridge objective in Equation 10.

5.2 Bagging

Bagging (bootstrap aggregating) reduces variance by averaging many trees, each fit to an independent bootstrap resample of the training rows:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x) \tag{13}$$

where each $\hat{f}_b$ is a tree fit to bootstrap sample b [1]. Random forests extend Equation 13 by additionally restricting each split in Equation 12 to a random subset of features, further decorrelating the individual trees [2].

5.3 Gradient boosting

Where bagging averages independent trees fit in parallel, gradient boosting fits trees sequentially, each one correcting the errors of the ensemble built so far. The model is an additive expansion

$$F_M(x) = \sum_{m=1}^{M} \gamma_m h_m(x) \tag{14}$$

built greedily: at each stage m, tree h_m is fit to the negative gradient (pseudo-residual) of the loss function L with respect to the current model's predictions,

$$r_{i,m} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F = F_{m-1}} \tag{15}$$

which for squared-error loss reduces to the ordinary residual $y_i - F_{m-1}(x_i)$. This is gradient descent carried out in the space of functions rather than parameters [7]. `HistGradientBoostingRegressor` (scikit-learn), used in this project's Session 4, implements Equations 14–15 with histogram-binned feature values, which reduces the split-search cost of Equation 12 from scanning every unique value to scanning a fixed number of bins, and handles missing feature values natively.

5.4 XGBoost

XGBoost augments gradient boosting with an explicit regularization term and a second-order (Newton) approximation of the loss. Its objective at boosting round m is

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} l(y_i,\, F_{m-1}(x_i) + h_m(x_i)) + \Omega(h_m), \qquad \Omega(h) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{16}$$

where T is the number of leaves in tree h_m and w its leaf weights. This project treats XGBoost as a comparison model, not the project's goal (see `AGENTS.md`), with a small, defensible hyperparameter grid selected on the validation years rather than a wide search, implemented in `notebooks/05_xgboost.ipynb` and `src/train.py`.

5.4.1 The second-order (Newton) step

Rather than fitting h_m to the first-order pseudo-residual alone (Equation 15), XGBoost approximates the loss in Equation 16 by its second-order Taylor expansion around the current prediction $F_{m-1}(x_i)$. Writing, for each observation,

$$g_i = \left.\frac{\partial\, l(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)}, \qquad h_i = \left.\frac{\partial^2 l(y_i, F)}{\partial F^2}\right|_{F=F_{m-1}(x_i)}, \tag{17}$$

(the Hessian h_i carries an observation subscript i; the tree h_m carries a round subscript m, so the two symbols do not collide), the objective becomes, up to a constant that does not depend on h_m,

$$\tilde{\mathcal{L}}^{(m)} = \sum_{i=1}^{n} \left[g_i\, h_m(x_i) + \tfrac{1}{2} h_i\, h_m(x_i)^2 \right] + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2. \tag{18}$$

A tree with fixed structure assigns every observation to exactly one leaf j with constant value w_j. Collecting the per-leaf sums $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$, where I_j is the set of observations in leaf j, Equation 18 separates across leaves:

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^{T} \left[G_j w_j + \tfrac{1}{2}\,(H_j + \lambda)\, w_j^2 \right] + \gamma T. \tag{19}$$

Each bracket is a one-dimensional quadratic in w_j, minimized in closed form:

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad \tilde{\mathcal{L}}^{(m)}_{\min} = -\tfrac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T. \tag{20}$$

5.4.2 Split gain and pruning

Because Equation 20 scores any fixed tree structure, the value of splitting one leaf into left and right children (L, R) is the reduction in the minimized objective, the split gain [4]:

$$\text{Gain} = \tfrac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma. \tag{21}$$

Splits are grown greedily by maximizing Equation 21 over candidate features and thresholds; any split whose gain is negative after the γ deduction is not made, so γ acts as a pruning threshold

on weak splits. (The xgboost implementation reports gain without the constant factor $\frac{1}{2}$, which does not change which split is chosen; verified against xgboost 3.3.0, whose reported gain is exactly twice Equation 21 at $\gamma = 0$.)

5.4.3 Specialization to squared error, and what λ does

Under the squared-error loss $l = \frac{1}{2}(y_i - F)^2$ used in this project, Equation 17 gives $g_i = F_{m-1}(x_i) - y_i = -r_i$ (the negative residual) and $h_i = 1$, so for a leaf containing n_j observations Equation 20 becomes

$$w_j^* = \frac{\sum_{i \in I_j} r_i}{n_j + \lambda} = \bar{r}_j \cdot \frac{n_j}{n_j + \lambda}, \tag{22}$$

where $\bar{r}_j$ is the mean residual in the leaf. Plain gradient boosting (Section on gradient boosting above) would output $\bar{r}_j$ itself; XGBoost multiplies it by $n_j/(n_j + \lambda)$, shrinking every leaf's correction toward zero and shrinking thin-evidence leaves (small n_j) hardest. This is the second-order machinery's practical effect on this project's data: corrections supported by few country-year rows are damped before they can overfit. Equations 20, 21, and 22 were verified numerically against xgboost 3.3.0 on a worked four-observation example (a single depth-one tree, $\lambda = 1$): the fitted leaf values equal both closed-form expressions exactly, and the split chosen is the gain-maximal one.

5.4.4 Shrinkage, subsampling, and early stopping

Each fitted tree enters the ensemble scaled by a learning rate $\eta \in (0, 1]$:

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x), \tag{23}$$

so a smaller η requires more boosting rounds M to reach the same fit; η and M trade off directly, and slow learning with more trees generalizes better at higher compute cost [7]. XGBoost additionally fits each tree on a random fraction of rows (`subsample`) and columns (`colsample_bytree`), which decorrelates successive trees in the same spirit as bagging [4].

The number of rounds M is not tuned by grid search in this project but chosen by early stopping: trees are added while the validation-set MAE improves, and boosting stops when it has not improved for a fixed patience window, freezing

$$M^* = \arg \min_{m \le M_{\max}} \mathrm{MAE}_{\mathrm{val}}(F_m). \tag{24}$$

Early stopping is model selection on the validation years, which this project's protocol allows; selecting anything by test-set performance is not. For the single test evaluation, each model is refit on the pooled training and validation rows (target years through 2018) with the tree count fixed at M^* and early stopping disabled, so the test years never influence any modeling choice, including the number of trees.

5.4.5 Why trees require the delta parameterization

A regression tree predicts a constant within each region of Equation 12, so an ensemble of trees can never predict a level outside the range of its training targets. A country whose emissions rise beyond anything seen in training saturates at the training ceiling and is systematically under-predicted in the level parameterization of Equation 6. Predicting the bounded year-over-year change $\delta_{c,t+1}$ and reconstructing the level from the known current value sidesteps this failure mode, which is why the tree-based models in this project are evaluated in both parameterizations.

6 Uncertainty and Significance of a Skill Score

The metrics of Section 2 are point estimates. A skill score of, say, -0.078 reports that a model's average error is 7.8 percent larger than persistence's on the evaluation sample, but it does not say whether that gap is distinguishable from zero or is within sampling noise. For a comparison to support a claim, the gap needs an interval and a significance statement. This section formalizes both, implemented as optional functions in `src/evaluate.py` that are deliberately not wired into the mandatory comparison table.

6.1 Loss differential

Let $e_i^{\mathrm{m}} = |y_i - \hat{y}_i^{\mathrm{m}}|$ and $e_i^{\mathrm{p}} = |y_i - \hat{y}_i^{\mathrm{p}}|$ be the absolute errors of a candidate model and of persistence on row i. Their difference

$$d_i = e_i^{\mathrm{p}} - e_i^{\mathrm{m}} \tag{25}$$

is positive when the model beats persistence on row i. The classical Diebold–Mariano test [5] assesses the null hypothesis $\mathbb{E}[d_i] = 0$ (equal predictive accuracy) using the mean and a serial-correlation-robust variance of $\{d_i\}$. The skill score of Equation 5 is a monotone reparameterization of the ratio of mean absolute errors, so testing whether skill differs from zero is the same question as testing whether the mean loss differential (Equation 25) differs from zero.

6.2 Why a row-level interval is wrong here

The Diebold–Mariano test was designed for a single time series of forecasts. This project's evaluation sample is a panel: each country contributes several years, and a country's own years are correlated (its emissions, and its errors, move together). Treating the n rows as independent would understate the sampling variance, because the effective number of independent observations is closer to the number of countries than to the number of rows. In addition, a few large emitters dominate the absolute error (Section 2), so the influence of individual countries is uneven. Both features call for resampling at the level of the country, not the row.

6.3 Country-clustered bootstrap interval

Let $\mathcal{C}$ be the set of countries in the evaluation sample, with $|\mathcal{C}| = C$. A cluster bootstrap [3], [6] draws C countries with replacement, pools all rows of the drawn countries, and recomputes the statistic. For bootstrap replicate $b = 1, \ldots, B$, let $\mathcal{C}^{(b)}$ be the multiset of drawn countries and $R^{(b)}$ the pooled rows; the resampled skill is

$$s^{(b)} = 1 - \frac{\sum_{i \in R^{(b)}} e_i^{\mathrm{m}}}{\sum_{i \in R^{(b)}} e_i^{\mathrm{p}}} \tag{26}$$

Resampling whole countries preserves the within-country dependence that a row-level resample would destroy. A $100(1-\alpha)$ percent confidence interval for the skill is the empirical interval of the replicates,

$$\Big[\; q_{\alpha/2}\Big(\{s^{(b)}\}\Big)\,,\; q_{1-\alpha/2}\Big(\{s^{(b)}\}\Big)\;\Big] \tag{27}$$

where q_τ is the τ empirical quantile. A two-sided bootstrap p-value for the null hypothesis that the model ties persistence ($s = 0$) is

$$p = 2\min\left(\frac{1}{B}\sum_{b=1}^{B}\mathbf{1}\{s^{(b)} \le 0\},\ \frac{1}{B}\sum_{b=1}^{B}\mathbf{1}\{s^{(b)} \ge 0\}\right), \tag{28}$$

clipped to $[0, 1]$. This is a bootstrap tail probability, not an exact analytic test, and should be reported as such.

6.4 Worked reading

On this project's validation years (targets 2015–2018, 153 countries), the linear-trend baseline has point skill -0.078 against persistence, but the country-clustered 95 percent interval (Equation 27) spans roughly $[-0.34, +0.20]$ with a bootstrap p-value near 0.7. The honest conclusion is therefore not "trend loses to persistence" but "trend and persistence are statistically indistinguishable on this sample": the point gap is well within sampling noise once the country-level dependence is accounted for. Reporting the interval, not only the point estimate, is what keeps the evaluation honest at the level of statistical significance, which is the same discipline this project already applies at the level of the baseline comparison itself.

7 Error Decomposition and Feature Importance

The comparison table of Section 2 reports one number per model per split. Error analysis (Session 6) decomposes that number along observable dimensions of the panel: prediction year, country, and emitter-size tier, so that a claim like "the model beats persistence" can be scoped to where it is actually true.

7.1 Conditional error and within-group skill

For any partition of the evaluation rows into groups $\mathcal{G}$ (years, countries, or size tiers), the conditional MAE of model m in group $G \in \mathcal{G}$ is

$$\mathrm{MAE}_G^{\mathrm{m}} = \frac{1}{|G|}\sum_{i \in G}\left|y_i - \hat{y}_i^{\mathrm{m}}\right|, \tag{29}$$

and the within-group skill compares each group against persistence's error in the same group,

$$\mathrm{Skill}_G = 1 - \frac{\mathrm{MAE}_G^{\mathrm{m}}}{\mathrm{MAE}_G^{\mathrm{p}}}, \tag{30}$$

so each tier or year is its own honest comparison. Because overall MAE is a row-share-weighted average of Equation 29 over groups, a positive overall skill (Equation 5) is compatible with negative skill in most groups when the error mass is concentrated in a few of them. On this project's test years, the headline skill of the best model lives almost entirely in the giant-emitter tier while every learned model loses to persistence among small emitters; the overall number alone would have hidden that scope.

7.2 Scale dependence of error rankings

Absolute errors (Equations 1–3) weight rows by the size of the miss in Mt; percentage errors (Equation 4) weight misses by the emitter's own level. The two views can rank the same countries in reverse: a giant emitter can produce one of the sample's largest absolute errors at one of its

smallest percentage errors, while a mid-size volatile emitter shows the opposite signature. This is the scale-dependence problem of forecast-accuracy measures discussed by [10], observed here first on the validation years and re-verified on the held-out test years before being used in any figure. Percentage errors additionally degenerate as $y_i \to 0$, which is why the threshold in Equation 4 is part of the metric's definition.

7.3 Permutation importance and correlated features

For a frozen fitted model $\hat{f}$ with evaluation-set error $L(\hat{f})$, the permutation importance of feature j is the increase in error when column j is randomly permuted across the evaluation rows, breaking its relationship to the target while preserving its marginal distribution [2]:

$$\mathrm{PI}_j = \mathbb{E}\Big[L\Big(\hat{f};\, X^{(\pi_j)}\Big)\Big] - L\Big(\hat{f};\, X\Big)\,, \tag{31}$$

estimated by averaging over several independent permutations π_j. When two features carry overlapping information, as this project's co2 level, lag, and rolling-mean columns do by construction, permuting either column alone leaves the model able to lean on the other, so both can receive small values of Equation 31 even when their shared signal is essential. Importances over correlated features are therefore read as group structure rather than a per-feature ranking; no feature is dropped or added in response to them here, since the configuration is frozen post-test.

References

[1] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[3] A. Colin Cameron, Jonah B. Gelbach, and Douglas L. Miller. Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3):414–427, 2008.

[4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016. Free preprint: arXiv:1603.02754.

[5] Francis X. Diebold and Roberto S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995.

[6] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.

[7] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[8] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

[9] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021. Free online textbook.

[10] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.